much: in other words and as a simplification, it tells us if by slightly increasing the value of `x`, `f(x)` also increases, or increases, or yet stays the same.

We can apply this to our optimization algorithm: for instance, in figure 16.13, if we computed the first derivative of the cost function at point x_0, we'd get a negative value that would tell us that `f` grows when `x` becomes smaller. Since we want to move towards smaller values of `f`, then we know that we should update `x` by assigning it a larger value.

We can repeat this step over and over, thus following a downhill path along the cost function's surface.

If this looks easy to compute, however, in multidimensional spaces this gets far more complicated: for a n-dimensional function `g`, the gradient of the function at a given point is a vector whose components are the partial derivatives of the function, computed in that point.

For instance, with a 2D domain (see figure 16.15 to visualize it), we define the partial derivative of `g(x,y)` along `x` as:

$$g'_x(x,y) = \frac{\partial g}{\partial x} = \lim_{h \to 0} \frac{g(x+h,y) - g(x,y)}{h}$$

And we define the gradient of g in a point $P_0=(x_0,y_0)$ as a vector with one column and two rows, whose components are the partial derivates of function `g` with respect to `x` and `y`, computed at point P_0: